

Engenharia de Software Moderna

Cap. 1 - Introdução

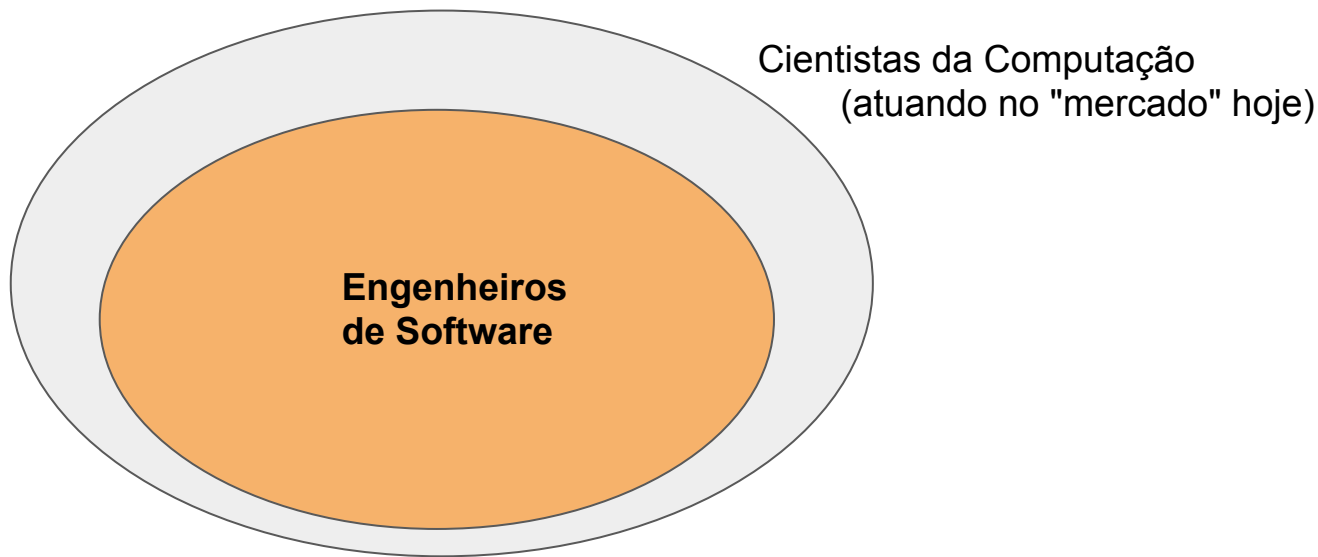
Prof. Marco Tulio Valente

<https://engsoftmoderna.info>, @engsoftmoderna

Licença CC-BY; permite copiar, distribuir, adaptar etc; porém, **créditos devem ser dados ao autor dos slides**

Importância do Curso

- No futuro, existe grande chance de você ser **Engenheiro de Software** (ou full stack developer, frontend developer, arquiteto, etc)



Conferência da OTAN (Alemanha, 1968)

- 1a vez que o termo Engenharia de Software foi usado



Working Conference on **Software Engineering**

Comentário de participante da Conferência da OTAN

"Certos sistemas estão colocando demandas que estão além das nossas capacidades... Em algumas aplicações não existe uma crise... Mas **estamos tendo dificuldades com grandes aplicações.**"

Definição de Engenharia de Software

Área da Computação destinada a investigar os desafios e propor soluções que permitam desenvolver sistemas de software — principalmente aqueles mais complexos e de maior tamanho — de forma produtiva e com qualidade

O que se estuda em ES?

1. Engenharia de Requisitos
2. Projeto de Software
3. Construção de Software
4. Testes de Software
5. Manutenção de Software
6. Gerência de Configuração
7. Gerência de Projetos



O que se estuda em ES? (cont.)

- 8. Processos de Software
- 9. Modelos de Software
- 10. Qualidade de Software
- 11. Prática Profissional
- 12. Aspectos Econômicos



Engenharia de Software Moderna

Cap. 2 - Processos

Prof. Marco Tulio Valente

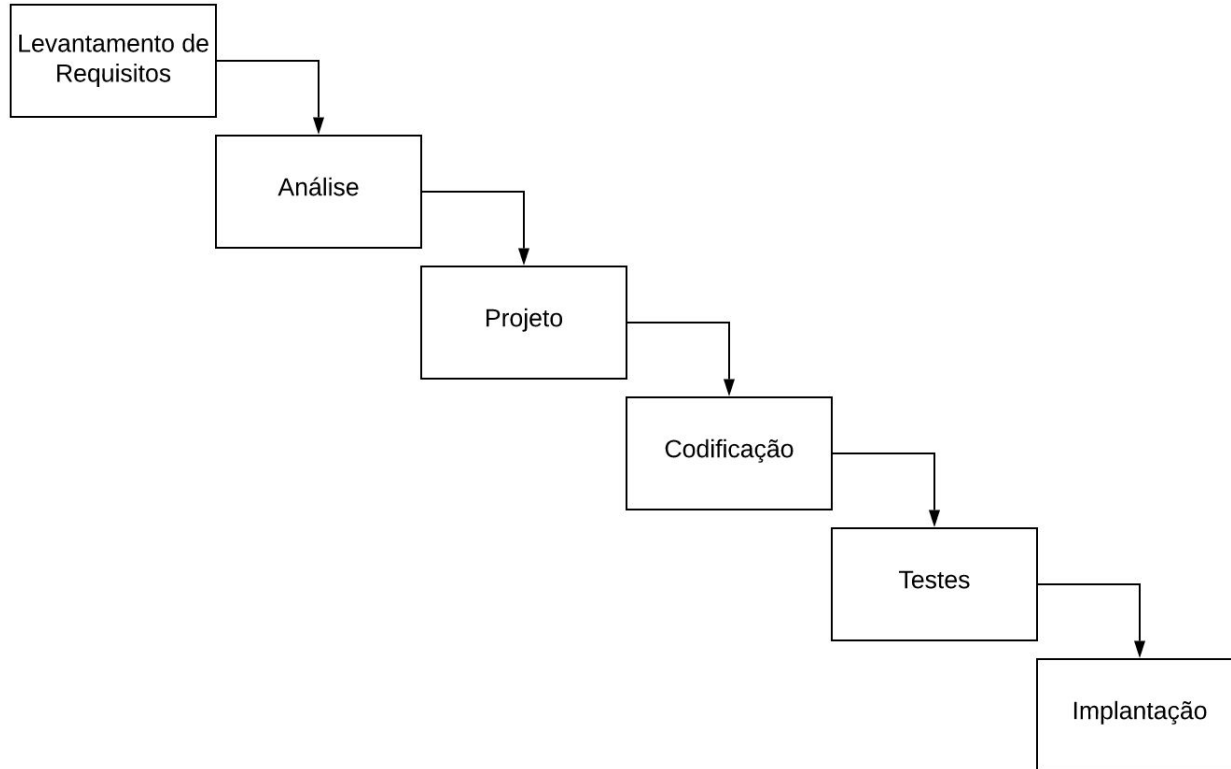
<https://engsoftmoderna.info>, @engsoftmoderna

Licença CC-BY; permite copiar, distribuir, adaptar etc; porém, **créditos devem ser dados ao autor dos slides**

Engenharia Tradicional

- Civil, mecânica, elétrica, aviação, automobilística, etc
- Projeto com duas características:
 - Planejamento detalhado (*big upfront design*)
 - Sequencial
- Isto é: Waterfall
 - Há milhares de anos

Natural que ES começasse usando Waterfall



No entanto: Waterfall não funcionou com software!

Software é diferente

- Engenharia de Software $\neq$ Engenharia Tradicional
- Software $\neq$ (carro, ponte, casa, avião, celular, etc)
- Software $\neq$ (produtos físicos)
- Software é abstrato e "adaptável"

Dificuldade 1: Requisitos

- Clientes não sabem o que querem (em um software)
 - Funcionalidades são "infinitas" (difícil prever)
 - Mundo muda!
- Não dá mais para ficar 1 ano levantando requisitos, 1 ano projetando, 1 ano implementando, etc
- Quando o software ficar pronto,
ele estará obsoleto!

Dificuldade 2: Documentações Detalhadas

- Verbosas e pouco úteis
- Na prática, desconsideradas durante implementação
- *Plan-and-document* não funcionou com software



Manifesto Ágil (2001)

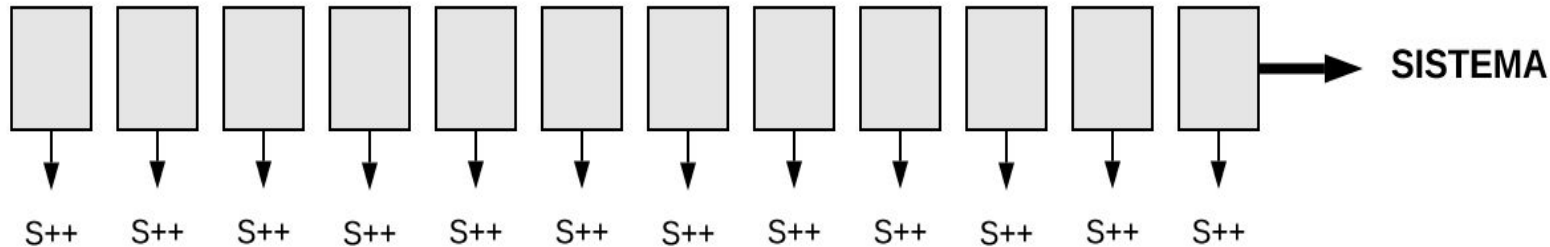


Ideia central: desenvolvimento iterativo

Waterfall



Ágil



Desenvolvimento iterativo

- Suponha um sistema imenso, complexo etc
- Qual o menor "incremento de sistema" eu consigo implementar em 15 dias e validar com o usuário?
- Validar é muito importante!
- Cliente não sabe o que quer!

Reforçando: **ágil = iterativo**

Outros pontos importantes (1)

- Menor ênfase em documentação
- Menor ênfase em *big upfront design*
- Envolvimento constante do cliente (*product owner*)

Outros pontos importantes (2)

- Novas práticas de programação
 - Testes, refactoring, integração contínua, etc

Métodos Ágeis

Métodos Ágeis

- Dão mais consistência às ideias ágeis
 - Definem um processo, mesmo que leve
 - Workflow, eventos, papéis, práticas, princípios etc

Métodos Ágeis que Vamos Estudar

- Extreme Programming (XP)
- Scrum
- Kanban

Antes de começar

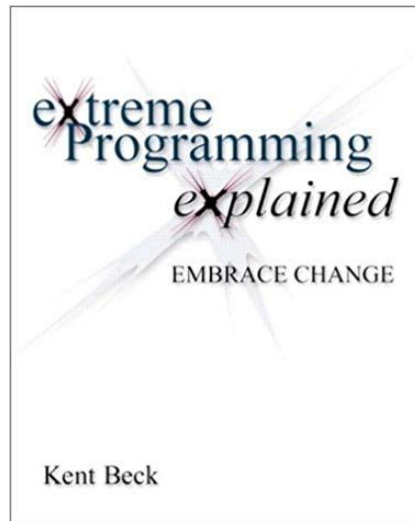
- Nenhum processo é uma bala-de-prata
- Processo ajuda a não cometer certos "grandes erros"
- Processos não são adotados 100% igual ao manual
 - Bom senso é importante
 - Experimentação é importante

Extreme Programming (XP)

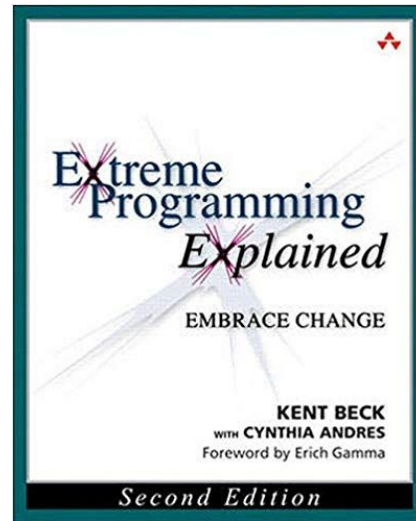
Extreme Programming



Kent Beck



1999



2004

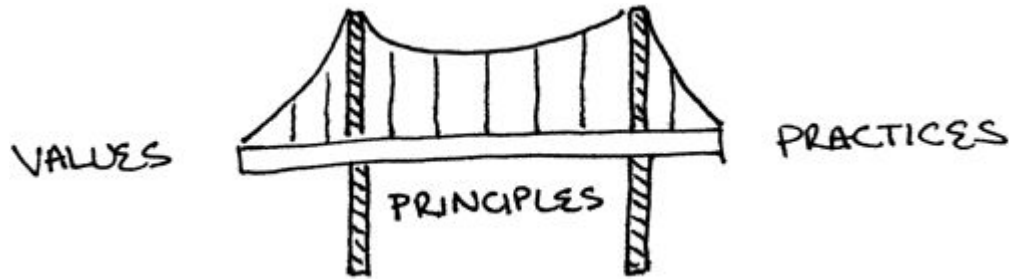
XP = Valores + Princípios + Práticas

Valores

- Comunicação
- Simplicidade
- Feedback
- Coragem
- Respeito
- Qualidade de Vida (semana 40 hrs)

Valores ou "cultura" são fundamentais em software!

XP = Valores + Princípios + Práticas



Princípios

- Economicidade
- Melhorias Contínuas
- Falhas Acontecem
- Baby Steps
- Responsabilidade Pessoal

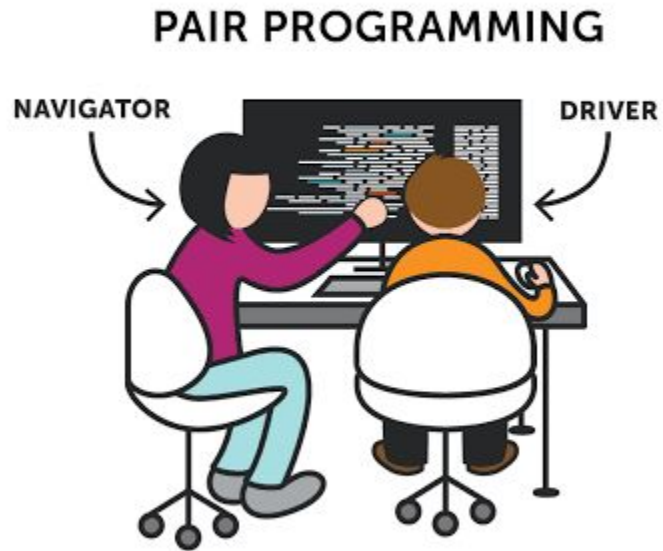
XP = Valores + Princípios + Práticas

Práticas sobre o Processo de Desenvolvimento	Práticas de Programação	Práticas de Gerenciamento de Projetos
Representante dos Clientes Histórias de Usuário Iterações Releases Planejamento de Releases Planejamento de Iterações Planning Poker Slack	Design Incremental Programação Pareada Testes Automatizados Desenvolvimento Dirigido por Testes (TDD) Build Automatizado Integração Contínua	Ambiente de Trabalho Contratos com Escopo Aberto Métricas

Iremos estudar em Scrum

Testes e TDD = Cap. 8

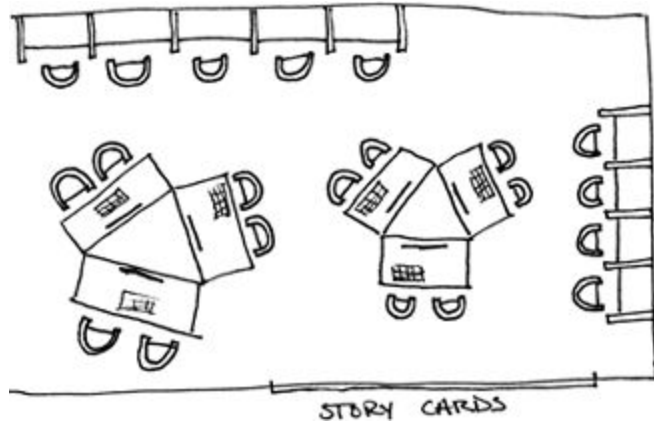
Pair Programming



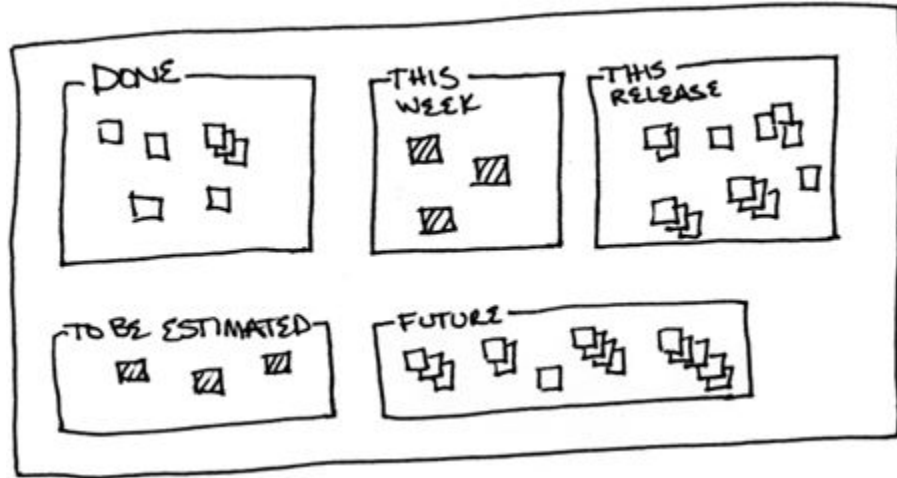
Estudo com Engenheiros da Microsoft (2008)

- Vantagens:
 - Redução de bugs
 - Código de melhor qualidade
 - Disseminação de conhecimento
 - Aprendizado com os pares
- Desvantagem:
 - Custo

(2) Ambiente de Trabalho



Ambiente de trabalho



Cartazes para "visualizar trabalho" em andamento

Contratos com Escopo Aberto

- Escopo fechado
 - Cliente define requisitos ("fecha escopo")
 - Empresa desenvolvedora: preço + prazo
- Escopo aberto
 - Escopo definido a cada iteração
 - Pagamento por homem/hora
 - Contrato renovado a cada iteração

Contratos com Escopo Aberto

- Exige maturidade e acompanhamento do cliente
- Vantagens:
 - Privilegia qualidade
 - Não vai ser "enganado" ("entregar por entregar")
 - Pode mudar de fornecedor

Scrum

Scrum

- Proposto por Jeffrey Sutherland e Ken Schwaber (OOPSLA 1995)

SCRUM Development Process

Ken Schwaber

Advanced Development Methods

131 Middlesex Turnpike Burlington, MA 01803

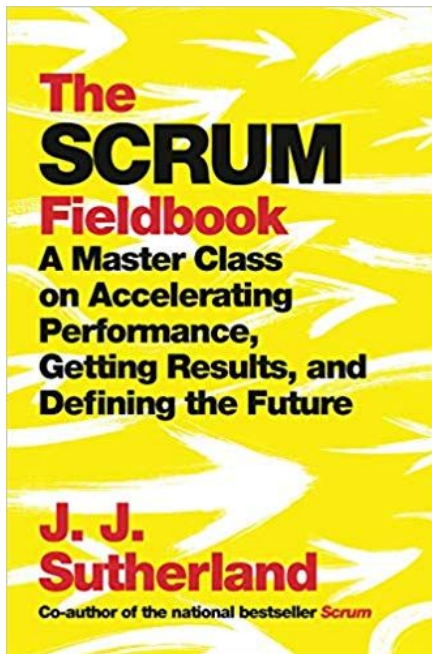
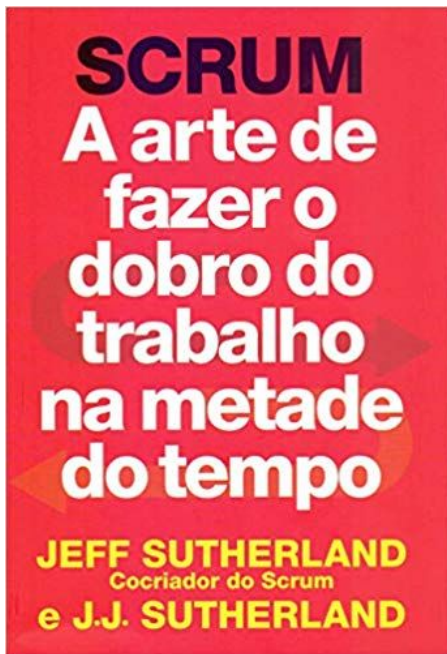
email virman@aol.com Fax: (617) 272-0555

ABSTRACT. *The stated, accepted philosophy for systems development is that the development process is a well understood approach that can be planned, estimated, and successfully completed. This has proven incorrect in practice. SCRUM assumes that the systems development process is an unpredictable, complicated process that can only be roughly described as an overall progression. SCRUM defines the systems development process as a loose set of activities that combines known, workable tools and techniques with the best that a development team can devise to build systems. Since these activities are loose, controls to manage the process and inherent risk are used. SCRUM is an enhancement of the commonly used iterative/incremental object-oriented development cycle.*

KEY WORDS: SCRUM SEI Capability-Maturity-Model Process Empirical

Scrum

- Scrum é uma indústria: livros, consultoria, certificações, marketing

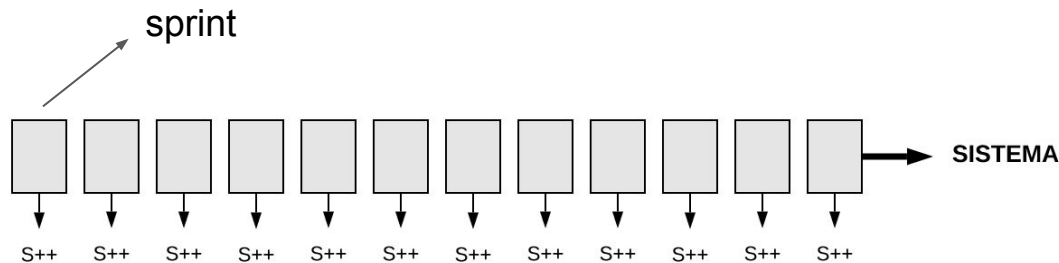


Scrum vs XP

- Scrum não é apenas para projetos de software
 - Logo, não define práticas de programação, como XP
- Scrum define um "processo" mais rígido que XP
 - Eventos, papéis e artefatos bem claros

Principal evento: Sprints

- Como em qualquer método ágil, desenvolvimento é dividido em sprints (iterações)
- Duração de um sprint: até 1 mês, normalmente 15 dias



O que se faz em um sprint?

- Implementa-se algumas **histórias dos usuários**
- Histórias = funcionalidades (ou features) do sistema
- Exemplo: fórum de perguntas e respostas

Postar Pergunta

Um usuário, quando logado no sistema, deve ser capaz de postar perguntas. Como é um site sobre programação, as perguntas podem incluir blocos de código, os quais devem ser apresentados com um layout diferenciado.

Bem simples, deve caber em um post-it

Quem escreve as histórias?

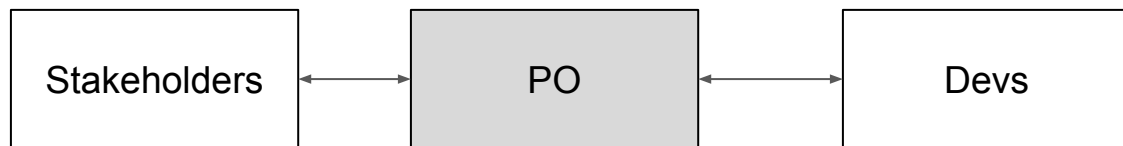
- **Product Owner (PO)**
- Membro (papel) obrigatório em times Scrum
- Especialista no domínio do sistema

Antes (Waterfall)



Observação: stakeholders são os clientes do sistema e qualquer outra parte interessada ou afetada por ele.
Exemplo: departamento jurídico é interessado no módulo de contratos de um sistema

Hoje (Scrum)



- Durante sprint, PO explica histórias (requisitos) para devs
- Troca-se documentação formal e escrita por documentação verbal e informal
- Conversas entre PO e Devs

Funções de um PO

- Escrever histórias dos usuários
- Explicar histórias para os devs, durante o sprint
- Definir "testes de aceitação" de histórias
- Priorizar histórias
- Manter o backlog do produto

Backlog do Produto

- Lista de histórias do usuário, que foram escritas pelo PO
- Duas características:
 - Priorizada: histórias do topo têm maior prioridade
 - Dinâmica: histórias podem sair e entrar, à medida que o sistema evolui

Resumindo

- Sprint (evento)
- PO e Devs (papeis)
- Backlog do produto (artefato)

Quais histórias vão entrar no próximo sprint?

- Essa decisão é tomada no início do sprint
- Em uma reunião chamada de **planejamento do sprint**
- PO propõe histórias que gostaria de ver implementadas
- Devs decidem se têm **velocidade** para implementá-las

Importante

- Em um time scrum, todos têm o mesmo nível hierárquico
- PO não é o "chefe" dos Devs
- Devs têm autonomia para dizer que não vão conseguir implementar tudo que o PO quer em um único sprint

Voltando ao Planejamento do Sprint

- 1a parte da reunião:
 - Definem-se as histórias do sprint
- 2a parte da reunião:
 - Histórias são quebradas em tarefas
 - Tarefas são alocadas a devs

Exemplo:

- História: Postar Perguntas
- Tarefas:
 - Projetar e testar a interface Web, incluindo layout, CSS templates, etc.
 - Instalar banco de dados, projetar e criar tabelas.
 - Implementar a camada de acesso a dados.
 - Implementar camada de controle, com operações para cadastrar, remover e atualizar perguntas.
 - Implementar interface Web.

Backlog do Sprint

- Lista de tarefas do sprint, com responsáveis e duração estimada

Sprint está pronto para começar!

Scrum: Conceitos Complementares

Times Scrum

- Pequenos, do tamanho de "duas pizzas"
- 5 a 11 membros
 - 1 PO
 - 3 a 9 desenvolvedores
 - 1 Scrum Master

Scrum Master

- Especialista em Scrum do time
 - Ajudar o time a seguir Scrum e seus eventos
- Removedor de "impedimentos" não-técnicos
 - Exemplo: desenvolvedores não têm máquinas boas
- Não é o chefe do time
- Pode pertencer a mais de um time

Tipos de Product Owner

- Sistema acadêmico
- Cliente/contratante: Pró-reitoria de Graduação
- #1: desenv. interno $\Rightarrow$ PO: funcionário da prograd
- #2: desenv. terceirizado $\Rightarrow$ PO: funcionário da prograd
- #3: compra produto $\Rightarrow$ PO: "proxy" do cliente
 - Vendas, marketing, etc
- PO: próximo do problema

Mais alguns eventos

Reuniões Diárias

- 15 minutos de duração; em pé. Cada participante diz:
 - o que ele fez ontem
 - o que pretende fazer hoje
 - e se está tendo alguma dificuldade
- Objetivos:
 - Melhorar comunicação
 - Antecipar problemas

Sprint termina com dois eventos:
Review e Retrospectiva

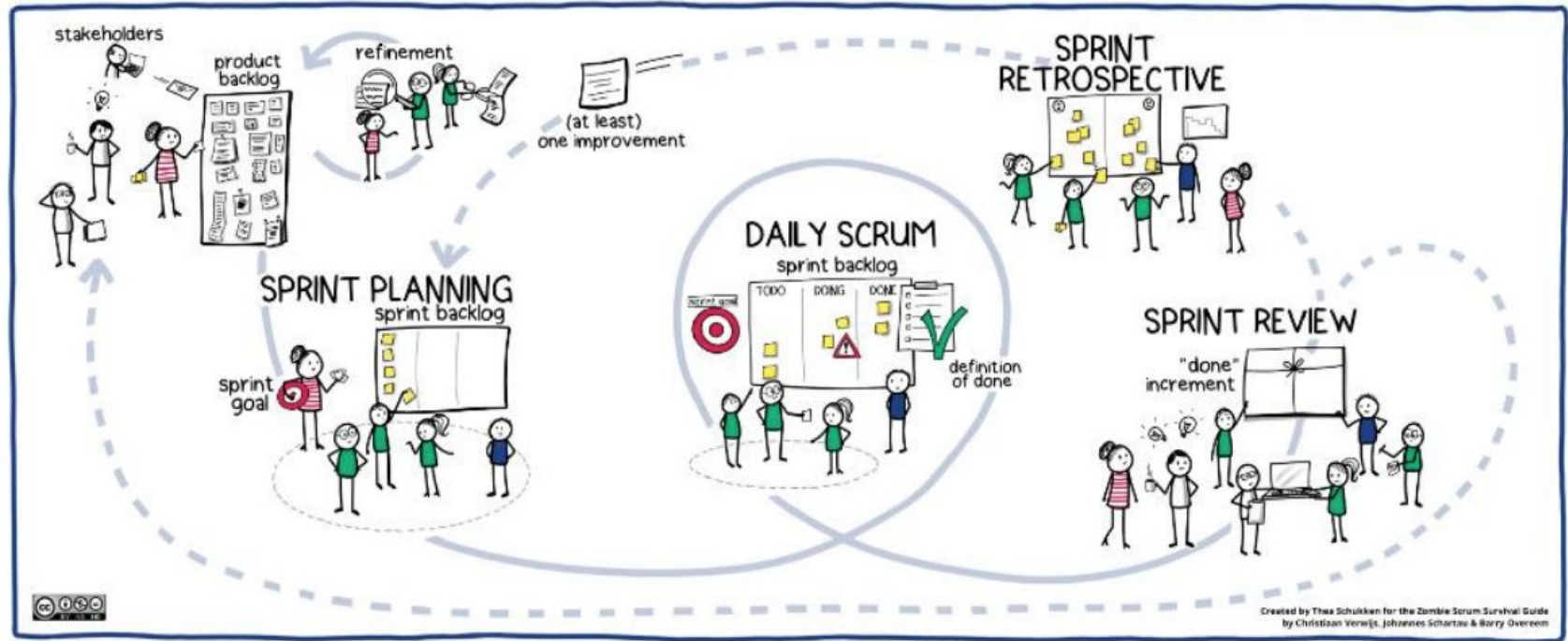
Revisão do Sprint

- Time mostra o resultado do sprint para PO e stakeholders
- Implementação das histórias pode ser:
 - Aprovada
 - Aprovada parcialmente
 - Reprovada
- Nos dois últimos casos, história volta para o backlog do produto

Retrospectiva

- Último evento do sprint
- Time se reúne para decidir o que melhorar
 - O que deu certo?
 - Onde precisamos melhorar?
- Modelo mental: melhorias constantes
- Não é para "lavar a roupa suja"

Resumo em 1 slide



Mais alguns conceitos de Scrum

Time-box

- Atividades têm uma duração bem definida

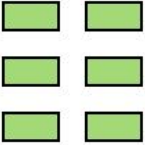
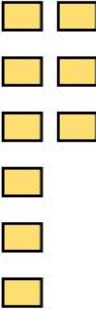
Evento	Time-box
Planejamento do Sprint	máximo de 8 horas
Sprint	menos de 1 mês
Reunião Diária	15 minutos
Revisão do Sprint	máximo de 4 horas
Retrospectiva	máximo de 3 horas

Critérios para Conclusão de Histórias (done criteria)

- Importante: considerar qualidade externa e interna
- Qualidade externa:
 - Testes de aceitação (caixa preta ou funcionais)
 - Testes não-funcionais (ex.: desempenho, usabilidade)
- Qualidade interna:
 - Testes de unidade
 - Revisão de código

Mais alguns artefatos de Scrum

Scrum Board

Backlog	To Do	Doing	Testing	Done
				

Scrum Board



Exemplo: projeto da Mozilla (usando GitHub Projects)

Smart Scheduling

Updated 13 days ago

Filter cards

100 Backlog

📄 Bug 1632870 - Store test configuration in the task definition

Added by ahal

📄 Bug 1667401 - Always schedule "new manifests" with manifest based bugbug optimizers

Added by ahal

📄 Investigate Treeherder UI/UX changes for test filtering

Added by armenzg

⚠️ Handle pushes containing multiple backouts

mozci#204 opened by marco-c

enhancement

⚠️ When a task/group has inconsistent classifications, use the status of the task/group on the backout to figure it out

mozci#200 opened by marco-c

enhancement

📄 Bug 1635921 - Use |mach try fuzzy| as a means to select configurations with

1 Next up

⚠️ Reinstate integration tests

mozci#390 opened by marco-c

3 In progress

📄 Build a dashboard to see schedulers results over time (both # of tests and durations)

Added by marco-c

⚠️ Add a way to get durations of shadow scheduler tasks

mozci#102 opened by ahal

metrics

1 linked pull request

📄 Bug 1639164 - "mach try auto" should select the best platforms to run manifests on

assigned: marco

Added by marco-c

222 Done

🔁 Make adr dependency optional

mozci#388 opened by marco-c

📄 Bug 1671422 - Add durations to group_result actions in errorssummary formatter

assigned: ahal

Added by ahal

🔁 Cache results from data sources

mozci#368 opened by marco-c

enhancement

1 linked pull request

🔁 OOMs while analyzing some pushes

mozci#366 opened by marco-c

bug

1 linked pull request

🔁 Add support for getting groups and groups results in the Treeherder data source

mozci#312 opened by marco-c

1 linked pull request

Exemplo: GitLab (https://gitlab.com/gitlab-org/gitlab/-/boards)

The image displays three GitLab Kanban boards, each representing a different workflow or project phase. Each board has a header with a title, a count of items, and a user icon. The tasks are represented as cards with titles, labels, and status indicators.

Board 1: Open (34472 items, 7577 users)

- Task 1:** Disable `ci\_disable\_validates\_dependencies` after 10.3 RC3 release
Labels: Category:Continuous Integration, devops, verify, group, continuous integration, section, ops
ID: #257852
- Task 2:** Update doc/development/pipelines.md for graphql-verify job
Labels: Engineering Productivity, documentation, missed:13.8
ID: #297030
- Task 3:** Discuss: Using Dynamic Child Pipelines in GitLab Pipeline
Labels: Engineering Productivity, ep, pipeline, meta
ID: #267491
- Task 4:** Consider dynamic analysis test mapping to streamline test execution
Labels: Engineering Productivity, meta
ID: #222369

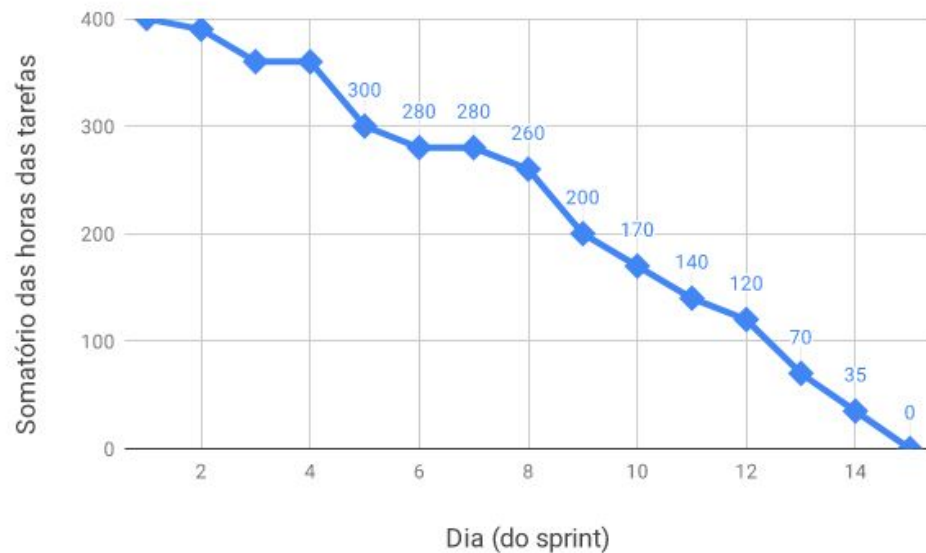
Board 2: workflow in dev (278 items, 418 users)

- Task 1:** Automated (pipeline) test planning for maintenance mode
Labels: Quality, auto updated, devops, enablement, geo, active, group, geo, missed-deliverable, missed:13.7, missed:13.8, section, enablement
ID: #266992
- Task 2:** Investigate: pipelines' behaviour in maintenance mode
Labels: Category:Disaster Recovery, Enterprise Edition, GitLab Premium, devops, enablement, geo, active, group, geo, section, enablement
ID: #296533
- Task 3:** BE: Retrieve details for individual Jira issues
Labels: Category:Integrations, Deliverable, Integration, Jira, atlassian, backend, customer, devops, create, feature, group, ecosystem, section, dev
ID: #294155
- Task 4:** JIRA integration doesn't use the authentication for the proxy provided.
Labels: Category:Integrations, Integration, Jira

Board 3: workflow in review (134 / 1 items, 201 users)

- Task 1:** Make it clearer what to do after adding a namespace
Labels: Category:Integrations, Integration, Jira, Jira, ConnectApp, Technical Writing, devops, create, frontend, group, ecosystem, missed:13.7, priority, 4, section, dev
ID: #235909
- Task 2:** Display feature flag information in Jira
Labels: Category:Integrations, Deliverable, Integration, Jira, Jira, ConnectApp, atlassian, backend, devops, create, feature, group, ecosystem, missed-deliverable, missed:13.8, priority, 1, section, dev
ID: #229347, 3 users
- Task 3:** Add a setting to allow/disallow duplicate Maven package uploads
Labels: Category:Package Registry, Deliverable, Package:P1, Stretch, backend, devops, package, feature, feature, addition, group, package, quad-planning, complete-action, ruby, section, ops
ID: #276882, 2 users

Burndown chart



Estimativa de Tamanho de Histórias de Usuários

Story points

- Unidade (inteiro) para comparação do tamanho de histórias. Exemplo de escala: 1, 2, 3, 5, 8, 13

História	Story Points
Cadastrar usuário	8
Postar perguntas	5
Postar respostas	3
Tela de abertura	5
Gamificar perguntas e respostas	5
Pesquisar perguntas e respostas	8
Adicionar tags em perguntas e respostas	5
Comentar perguntas e respostas	3

Velocidade de um time

- Número de story points que o time consegue implementar em um sprint
- Definição de story points é "empírica"

Kanban

Kanban

- Origem na década de 50 no Japão
- Sistema de Produção da Toyota
- Manufatura Lean
- Produção just-in time

Kanban = "cartão visual"



Kanban em Desenvolvimento de Software



Kanban vs Scrum

- Kanban é mais simples
- Não existem sprints
- Não é obrigatório usar papéis e eventos, incluindo:
 - Scrum master
 - Daily Scrum, Retrospectivas, Revisões
 - etc
- Time define os papéis e eventos

Kanban

"Grandes" colunas do quadro:
Passos

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas	em implementação	implementadas	em revisão	revisadas

1a sub-coluna:
em andamento

2a sub-coluna:
concluídas

Fluxo de trabalho (tempo)

Kanban

- Ideia central: sistema pull
- Membros "puxam" trabalho:
 - a. Escolhem uma tarefa para trabalhar
 - b. Concluem tarefa (movem ela para frente no quadro)
 - c. Volte para o passo (a)



tempo

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas	em implementação	implementadas	em revisão	revisadas



Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
☐	em espec.	especificadas	em implementação	implementadas	em revisão	revisadas

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas	em implementação	implementadas	em revisão	

tempo

tempo

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
☐	em espec.	especificadas	em implementação	implementadas	em revisão	revisadas

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas	em implementação	implementadas	em revisão	

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas	em implementação	implementadas	em revisão	revisadas



tempo

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas □ □ □ □	em implementação	implementadas	em revisão	revisadas

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas □ □ □	em implementação □	implementadas	em revisão	revisadas

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
	em espec.	especificadas □ □ □	em implementação	implementadas □	em revisão	

Exemplo 2

Exemplo 2: Ontem no final do dia

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
H3	em espec. H2	especificadas T6 T7 T8 T9	em implementação T4 T5	implementadas T3	em revisão T2	revisadas T1

Hoje no final do dia:

Backlog	Especificação WIP		Implementação WIP		Revisão de Código WIP	
H3	em espec.	especificadas T8 T9 T10 T11 T12	em implementação T4 T5 T6 T7	implementadas	em revisão T3	revisadas T1 T2

Kanban: Limites WIP

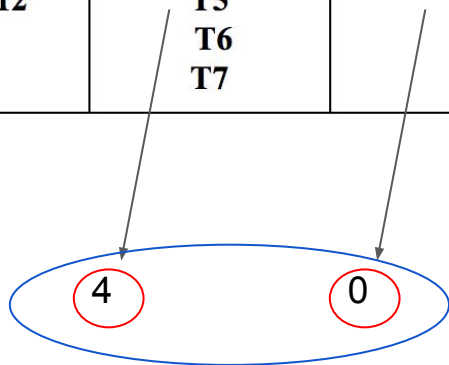
Limites WIP (Work in Progress)

Backlog	Especificação		Implementação		Revisão de Código	
		2		5	3	
H3	em espec.	especificadas T8 T9 T10 T11 T12	em implementação	implementadas T4 T5 T6 T7	em revisão T3	revisadas T1 T2

Limites WIP (Work in Progress)

- Número máximo de tarefas em um passo
- Contando: tarefas em andamento e concluídas

Backlog	Especificação 2		Implementação 5		Revisão de Código 3	
H3	em espec.	especificadas T8 T9 T10 T11 T12	em implementação T4 T5 T6 T7	implementadas	em revisão T3	revisadas T1 T2



Objetivos dos Limites WIP

- Criar um fluxo de trabalho sustentável
 - Evitar que o time fique sobrecarregado de trabalho
 - WIP = "acordo" entre o time e a organização
 - Capacidade de trabalho de um time
- Evitar que o trabalho fique concentrado em um passo

Exemplo de **violação** de WIP

Backlog	Especificação (2)		Implementação (5)		Validação (3)	
			X	X		
			X	X		
			X	X		

- Esse quadro não é válido
- Existem 6 tarefas em Implementação
- Mas o WIP desse passo é 5

Mais um exemplo: Implementação no limite

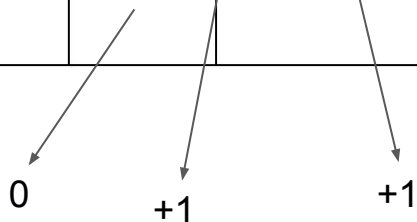
Backlog	Especificação (2)		Implementação (5)		Revisão (3)	
			X X	X X X		

- Talvez seja o momento de revisar algumas tarefas

WIP do Passo de Especificação

- Número de histórias em especificação + número de grupos de tarefas especificadas (linhas do quadro)
- Tarefas na mesma linha = resultantes da mesma história

Backlog	Especificação 2		Implementação 5		Revisão de Código 3	
H3	em espec.	especificadas	em implementação	implementadas	em revisão	revisadas
		T8 T9 T10 T11 T12	T4 T5 T6 T7		T3	T1 T2



WIP de Revisão de Código

- Conta apenas tarefas na primeira coluna (em revisão)

Backlog	Especificação 2		Implementação 5		Revisão de Código 3	
H3	em espec.	especificadas T8 T9 T10 T11 T12	em implementação T4 T5 T6 T7	implementadas	em revisão T3	revisadas T1 T2

1

Não contam para
fins de WIP

Comentários Finais sobre Kanban

- Kanban é mais simples do que Scrum
- Kanban é mais adequado para times mais maduros
- Talvez: começar com Scrum e depois migrar para Kanban

Quando não usar métodos ágeis?

Quando **não** vale a pena usar métodos ágeis

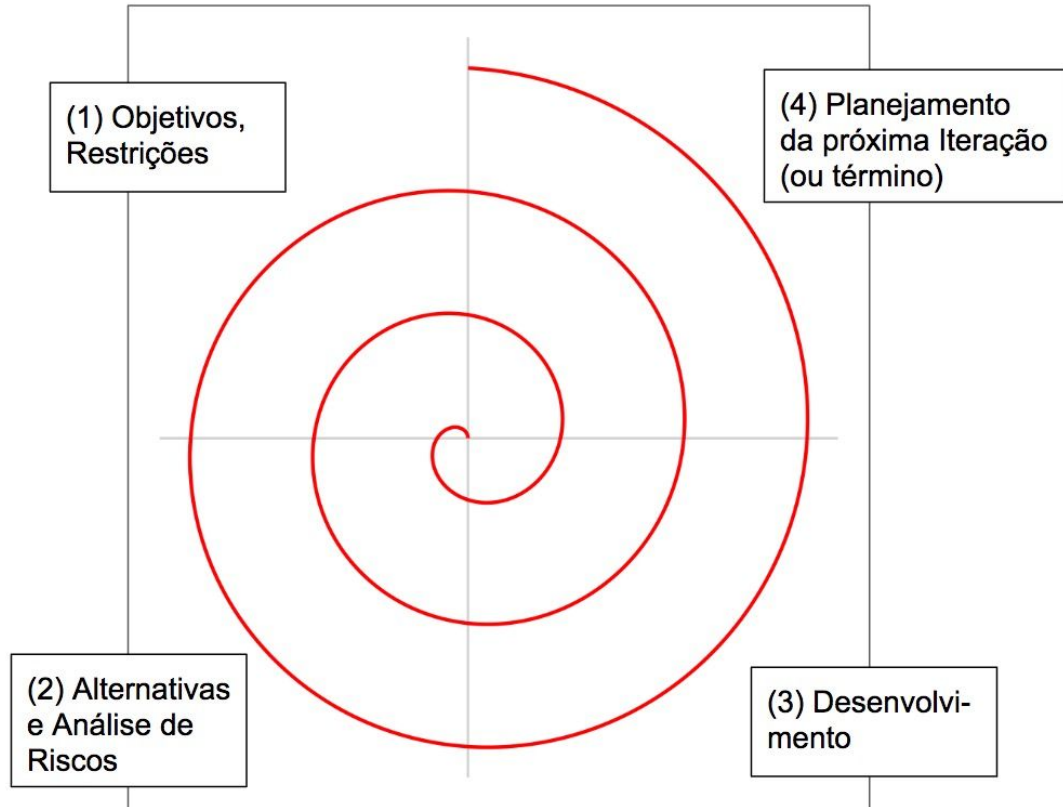
- Condições do mercado são estáveis e previsíveis
- Requisitos são claros no início do projeto e irão permanecer estáveis
- Clientes não estão disponíveis para colaboração frequente
- Sistema semelhante já foi feito antes; logo, já se conhece a solução
- Problemas podem ser resolvidos sequencialmente, em silos funcionais
- Clientes não conseguem testar partes do produto, antes de completo
- Mudanças no final do projeto são caras ou impossíveis
- Impacto de mudanças provisórias pode ser catastrófico

Outros Processos (não ágeis)

Transição de Waterfall para Ágil

- Antes da disseminação dos princípios ágeis, alguns métodos **iterativos** ou **evolucionários** foram propostos
- Transição Waterfall (~1970) e Ágil (~2000) foi gradativa
- Exemplos:
 - Espiral (1986)
 - Rational Unified Process (RUP) (2003)

Modelo em Espiral



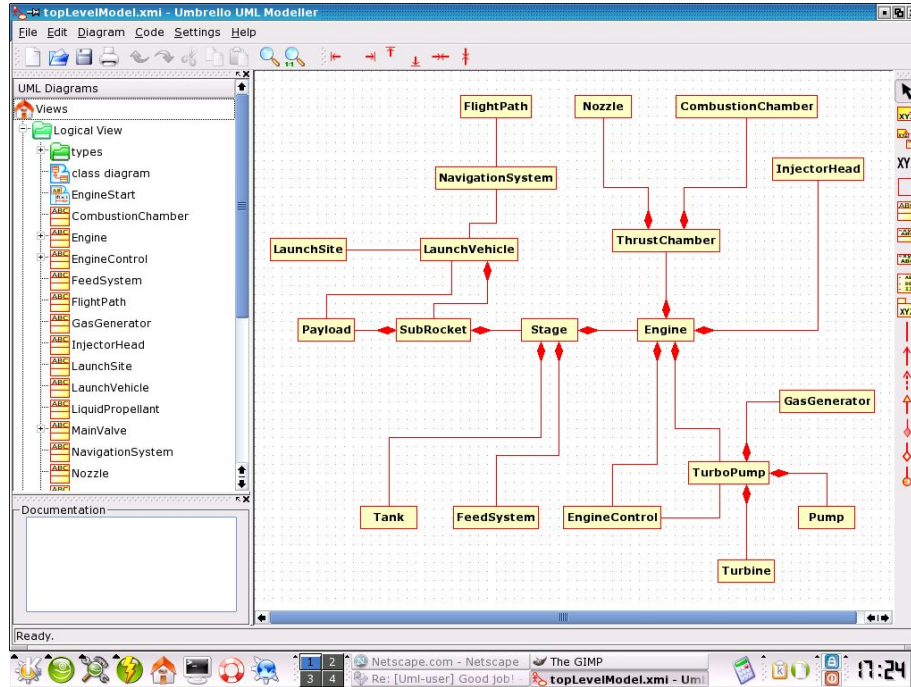
Proposto por Barry Boehm

Iterações: 6 a 24 meses (logo, mais que em XP ou Scrum)

Rational Unified Process (RUP)

- Proposto pela Rational, que depois comprada pela IBM
- Duas características principais:
 - Diagramas UML
 - Ferramentas CASE

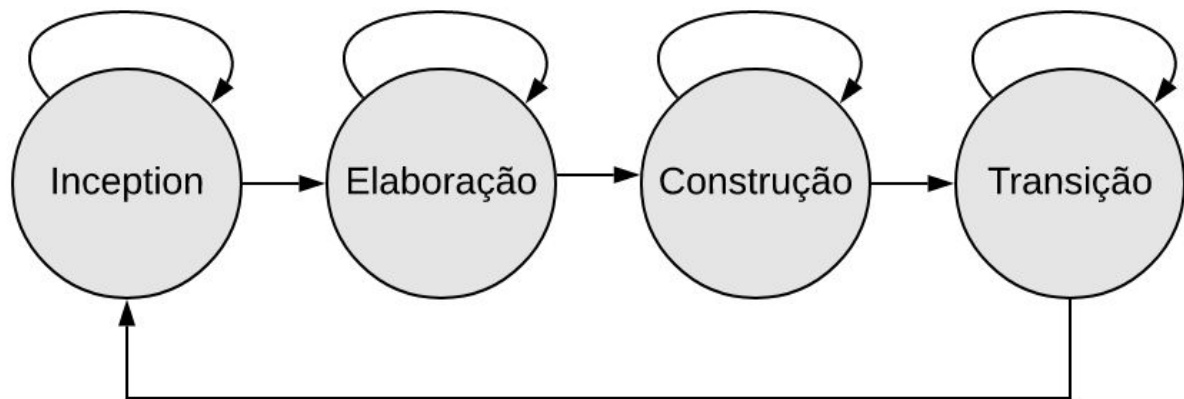
CASE (Computer-Aided Software Engineering)



Nome vem de sistemas de CAD
(usados em engenharia tradicional)



Fases do RUP



- Inception: análise de viabilidade
- Elaboração: requisitos + arquitetura
- Construção: projeto de baixo nível + implementação
- Transição: implantação (deployment)

FIM